Announcements 📣

- Project Phase #2 is due this Saturday at 11:59 PM
- Individual assignment 1 released due end this week Saturday at 11:59 PM
- Commit each TODO in the demo to the repository

Web Engineering & Development (SWE 363)

JavaScript Fundamentals I

In today's lecture:

- JS Syntax and Variables
- JS Arithmetic
- JS Conditionals
- JS Loops
- JS Functions
- JS Scope
- JS Arrays

Reference:

- Zybook: 4.1 to 4.8

What is JavaScript?

- **High-level**, **interpreted** programming language
- Adds **interactivity** and **dynamic behavior** to web pages
- Runs in the **browser** (client-side) and **server** (Node.js)
- One of the three core technologies of the web:
  - HTML: Structure
  - CSS: Presentation
  - **JavaScript: Behavior**

Syntax and Variables

Basic Syntax

```
// Single line comment

/* Multi-line
   comment */

// Statements end with semicolon (optional but recommended)
console.log("Hello, World!");
```

Variables: Declaring Storage

Three ways to declare variables:

```
var x = 5;          // Old way (function-scoped)
let y = 10;         // Modern way (block-scoped)
const z = 15;       // Constant (cannot be reassigned)
```

Key Differences:

- `let` : Value can change, block-scoped
- `const` : Value cannot be reassigned, block-scoped
- `var` : Avoid in modern code (function-scoped, hoisting issues)

Variable Naming Rules

```
// Valid names
let userName = "Alice";
let user_age = 25;
let $price = 99.99;
let _temp = 0;

// Invalid names
let 2users = 5;       // Cannot start with number
let user-name = "";   // Cannot use hyphen
let class = "CS";     // Cannot use reserved keywords
```

Convention: Use camelCase for variables and functions

Data Types

```
let name = "John";              // String
let age = 30;                   // Number
let isStudent = true;           // Boolean
let nothing = null;             // Null
let notDefined;                 // Undefined

// Check type
console.log(typeof age);     // "number"
console.log(typeof name);    // "string"
```

Arithmetic Operations

```
let x = 10;
let y = 3;

console.log(x + y);    // 13 (addition)
console.log(x - y);    // 7  (subtraction)
console.log(x * y);    // 30 (multiplication)
console.log(x / y);    // 3.333... (division)
console.log(x % y);    // 1  (modulus/remainder)
console.log(x ** 2);   // 100 (exponentiation)
```

Assignment Operators

```
let x = 10;

x += 5;   // x = x + 5  → 15
x -= 3;   // x = x - 3  → 12
x *= 2;   // x = x * 2  → 24
x /= 4;   // x = x / 4  → 6
x %= 4;   // x = x % 4  → 2

// Increment and Decrement
x++;      // x = x + 1
y--;      // y = y - 1
```

Conditionals: if Statements

```
let age = 20;

if (age >= 18) {
    console.log("You are an adult");
}

// if-else
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

Comparison Operators

```
let x = 5;
let y = "5";

console.log(x == y);   // true (loose equality, converts types)
console.log(x === y);  // false (strict equality, checks type too)
console.log(x != y);   // false
console.log(x !== y);  // true

console.log(x > 3);    // true
console.log(x <= 5);   // true
```

Best Practice: Always use `===` and `!==`

Logical Operators

```
let age = 25;
let hasLicense = true;

// AND operator
if (age >= 18 && hasLicense) { console.log("Can drive"); }

// OR operator
if (age < 18 || !hasLicense) { cons }

// NOT operator
if (!hasLicense) { console.log("Need a license"); }
```

Switch Statements

```
let day = 3;
let dayName;

switch (day) {
    case 1: dayName = "Monday";
            break;
    case 2: dayName = "Tuesday";
            break;
    case 3: dayName = "Wednesday";
            break;
    default: dayName = "Unknown";
}
console.log(dayName);  // "Wednesday"
```

Ternary Operator

Shorthand for simple if-else:

```
// Traditional if-else
let age = 20;
let status;

// Ternary operator
let status = age >= 18 ? "adult" : "minor";

// Another example
let price = 100;
let discount = price > 50 ? 0.2 : 0.1;
```

Loops: while Loop

```
// while loop
let count = 0;
while (count < 5) {
    console.log(count);
    count++;
}
// Output: 0 1 2 3 4

// do-while loop (runs at least once)
let x = 0;
do {
    console.log(x);
    x++;
} while (x < 3);
// Output: 0 1 2
```

for Loop

```
// Traditional for loop
for (let i = 0; i < 5; i++) {
    console.log(i);
}
// Output: 0 1 2 3 4

// Loop through array
let fruits = ["apple", "banana", "orange"];
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

Loop Control: break and continue

```
// break: exit loop immediately
for (let i = 0; i < 10; i++) {
    if (i === 5) break;
    console.log(i);
}
// Output: 0 1 2 3 4

// continue: skip current iteration
for (let i = 0; i < 5; i++) {
    if (i === 2) continue;
    console.log(i);
}
// Output: 0 1 3 4
```

Functions

Functions are **reusable blocks of code**

```
// Function declaration
function greet(name) {
    return "Hello, " + name + "!";
}

console.log(greet("Alice"));  // "Hello, Alice!"

// Function with multiple parameters
function add(a, b) {
    return a + b;
}

console.log(add(5, 3));  // 8
```

Function Parameters

```
// Default parameters
function greet(name = "Guest") {
    return `Hello, ${name}!`;
}

console.log(greet("Alice"));  // "Hello, Alice!"
console.log(greet());         // "Hello, Guest!"

// Multiple parameters
function calculatePrice(price, quantity = 1, tax = 0.1) {
    return price * quantity * (1 + tax);
}

console.log(calculatePrice(100, 2, 0.15));  // 230
```

Function Expressions and Arrow Functions

```
// Function expression
const multiply = function(x, y) {
    return x * y;
};

// Arrow function (ES6)
const divide = (x, y) => {
    return x / y;
};

// Shorter arrow function for single expression
const square = x => x * x;

console.log(square(5));  // 25
```

Scope

Scope determines where variables are accessible

```
// Global scope
let globalVar = "I'm global";

function myFunction() {
    // Function scope
    let localVar = "I'm local";
    console.log(globalVar);  // Accessible
    console.log(localVar);   // Accessible
}

myFunction();
console.log(globalVar);  // Accessible
console.log(localVar);   // Error! Not accessible
```

Block Scope

```
// Block scope with let and const
if (true) {
    let blockVar = "Block scoped";
    const blockConst = 10;
    var functionVar = "Function scoped";
}

console.log(functionVar);  // Accessible (var is function-scoped)
console.log(blockVar);     // Error! (let is block-scoped)
console.log(blockConst);   // Error! (const is block-scoped)
```

Tip: Use `let` and `const` to avoid scope issues

The Global Object

In browsers, the global object is `window`

```
// Global variables become properties of window
var x = 5;
console.log(window.x);  // 5

// Built-in global functions
console.log(window.parseInt("123"));  // 123
console.log(window.isNaN("hello"));   // true

// Access document, navigator, etc.
console.log(window.document);
console.log(window.navigator);
```

Arrays

Arrays store **ordered collections** of values

```
// Creating arrays
let fruits = ["apple", "banana", "orange"];
let numbers = [1, 2, 3, 4, 5];
let mixed = [1, "hello", true];

// Accessing elements (zero-indexed)
console.log(fruits[0]);  // "apple"
console.log(fruits[2]);  // "orange"

// Array length
console.log(fruits.length);  // 3
```

Modifying Arrays

```
let fruits = ["apple", "banana"];

// Add elements
fruits.push("orange");      // Add to end: ["apple", "banana", "orange"]
fruits.unshift("mango");    // Add to start: ["mango", "apple", "banana", "orange"]

// Remove elements
fruits.pop();               // Remove from end
fruits.shift();             // Remove from start

// Modify element
fruits[0] = "grape";
```

Iterating Over Arrays

```
let fruits = ["apple", "banana", "orange"];

// Traditional for loop
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}

// for...of loop (modern, cleaner)
for (let fruit of fruits) {
    console.log(fruit);
}

// forEach method
fruits.forEach(function(fruit) {
    console.log(fruit);
});
```

Common Array Methods

```
let numbers = [1, 2, 3, 4, 5];

// Find elements
console.log(numbers.indexOf(3));     // 2 (index)
console.log(numbers.includes(4));    // true

// Slice (extract portion)
let sliced = numbers.slice(1, 3);    // [2, 3]

// Splice (add/remove elements)
numbers.splice(2, 1);                // Remove 1 element at index 2

// Join (convert to string)
console.log(numbers.join(", "));     // "1, 2, 4, 5"
```

Array Transformation Methods

```
let numbers = [1, 2, 3, 4, 5];

// map: transform each element
let doubled = numbers.map(function(n) {
    return n * 2;
});
console.log(doubled);  // [2, 4, 6, 8, 10]

// filter: select matching elements
let evens = numbers.filter(function(n) {
    return n % 2 === 0;
});
console.log(evens);  // [2, 4]
```

Key Takeaways

1. **Variables**: Use `let` and `const` (avoid `var`)
2. **Conditionals**: `if`, `else if`, `else`, `switch`, ternary operator
3. **Loops**: `for`, `while`, `do-while`, `for...of`
4. **Functions**: Reusable code blocks with parameters and return values
5. **Scope**: Understand global, function, and block scope
6. **Arrays**: Ordered collections with powerful built-in methods

30m

Demo

4.1 JS Basics

Next Class

- More on JavaScript Fundamentals